

26A – M6 POS & Offline: Implementation Guide

Everything the till track needs: the Android project structure, the local database as Room schema, the issuance transaction, the sync worker, snapshot application, the scan layer, the peripheral layer with its simulator, the receipt renderer, sessions and PIN, modes and training, telemetry, the central-side package, tests including the power-loss harness, and the tickets. Self-contained; restated material is marked.

Field	Value
Document	26A – implementation guide for 26 M6 POS & Offline v0.1
Version	v0.1 – for the till track (two to three developers) and the M6 central developer
Date	9 September 2026
Prerequisite	17A till scaffold and shared engine; 19A sync gateway; 23A engine 1.0.0; the bundle and snapshot contracts of 24A §7 and 25A §7; the terminal model shortlist (doc 14 DR-3) for spikes 2, 3, 6.
Estimated effort	About 60 developer-days for the till (three developers, about four weeks), plus 8 for the central package
Restated content	Marked <i>restated from doc X §Y</i> .

Read this first. Build in this order and do not skip ahead: (1) the local database and the issuance transaction with the power-loss harness, (2) the sync worker against the central simulator until the thirteen conformance cases pass, (3) the sales screen on the simulator peripherals, (4) real peripherals, (5) shop mode, (6) everything else. A till that sells beautifully but loses a receipt on a power cut has failed; a till that syncs perfectly but takes five taps to sell will be worked around with a paper book.

1. What you are building

Restated from doc 26 §1. A native Android application that sells, records tenders, prints trilingual receipts, deducts stock, runs sessions with blind close, hosts shop-management screens, works two days offline, and synchronises without merging; plus a thin central package that ingests, re-validates and watches the fleet.

You build	You do not build
The till application: modules app, core, data, engine-adapter, sync, peripherals, ui; the training namespace; telemetry	The management agent (doc 31: open-source MDM); the identity provider; the kernel gateway (19A)
Shop-mode screens from the M4 and M5 specifications; the item lookup from M2's LookupResult; the picker from M3's candidates	The business rules inside those screens (validators come as data in the snapshot or as engine code)
Central package m6pos: receipt and session ingestion hooks, cross-session refund, sync health and fleet queries	M5 deductions, M7 account postings, M8 projections (they consume your events)

2. What you inherit

From	You use	How
17A §8 till scaffold	Project layout, Compose skeleton, Room mirror tables, fonts, simulator peripherals, HelloScreen	Extend; do not restart
17A §9 / 23A §6 shared engine	PriceResolver, RuleEvaluator, BasketResolver, PricingSnapshotIndex, Tax, Rounding, Envelope, Counter as a Maven artefact 1.0.0	Link; never fork
19A §8 sync gateway	Endpoints enrol, batches, snapshot, heartbeat, presign; ack and instruction semantics	Section 6
Doc 32	Batch limits, retention, gap handling, conformance suite, central simulator	Sections 6 and 10
19A §2, §3	Device credential auth; signed operator snapshot with Argon2id PIN hashes and offline permissions	Section 8
Doc 18	Table shapes for document, line, tender, state history, movement, audit, counters	Section 3
Doc 24A §7.3–7.4, doc 25A §7.3–7.4	Expected drop and transfer rows, count tasks, policies; GRN, count, write-off draft, repack, transfer receipt bundle shapes	Sections 3 and 9
Doc 22A §5, doc 23A §7.2	LookupResult shape; pricing snapshot rows	Sections 4 and 5
Doc 30 §2, §4	Design system, zones, usability rules, component vocabulary	Section 9

3. Local database (Room)

Restated from doc 18 and doc 26 §3.1. Two groups of tables: till-owned (written here, uploaded) and cache (received, replaced atomically). Entities are Kotlin data classes with Room annotations; the DDL below is the generated schema's meaning, not its exact text.

```
-- till-owned
document(document_id TEXT PK, doc_type_code, series_id, doc_number INTEGER, doc_number_display,
  owner_entity_id, counterparty_entity_id,
  location_id, till_position_id, device_id, status, issued_at, issued_local, business_date,
  operator_user_id, currency,
  net_amount, tax_amount, gross_amount, reference_document_id, content_hash, origin='OFFLINE',
  device_seq, bill_rule_id,
  bill_discount, rounding, engine_version, snapshot_version, notes, training INTEGER DEFAULT 0)
document_line(line_id PK, document_id, line_no, sku_id, batch_id, uom_code, qty, unit_price, mrp_applied,
  control_price_applied,
  cap_reason, discount_rule_id, discount_amount, tax_rate_percent, tax_amount, line_total,
  unit_cost_at_issue NULL,
  loss_category NULL, reference_line_id NULL, negative_ack INTEGER DEFAULT 0, picked INTEGER DEFAULT 0,
  weight_kg NULL)
doc_receipt_tender(receipt_id, seq, kind, amount, reference, customer_account_id, change_given, offline
  INTEGER)
document_state_history(history_id PK, document_id, from_status, to_status, occurred_at, occurred_local,
  actor_user_id, reason_code, reason_text)
document_link(from_document_id, to_document_id, link_type, amount)
stock_movement(movement_id PK, movement_seq INTEGER, location_id, batch_id, condition, qty_delta,
  movement_type, unit_cost_at_movement NULL,
  document_id, document_line_id, occurred_at, occurred_local, operator_user_id, device_id)
audit_event(audit_id PK, event_type_code, occurred_at, occurred_local, owner_entity_id, location_id,
  actor_user_id, device_id,
  till_position_id, subject_table, subject_id, document_id, before_state, after_state, reason_code,
  reason_text, device_seq, correlation_id)
till_session(session_id PK, till_position_id, operator_user_id, business_date, opened_at, float_amount,
  closed_at, counted_cash, expected_cash, variance, closed_by)
cash_movement(id PK, session_id, kind, amount, at, operator_user_id, reason)
parked_sale(id PK, lines_json, parked_at, operator_user_id)
counter(series_id PK, scope, next_number INTEGER) -- till-position series;
  location series when primary
outbox(device_seq INTEGER PK, event_id, event_type, payload BLOB, created_at, acked_at NULL) -- dense;
  retained 7 days after ack
sync_state(id=1, device_id, till_position_id, is_primary, last_acked_seq, snapshot_version,
  pending_snapshot_version, last_heartbeat_at,
  clock_offset_ms, floor_state, floor_grace_until, business_date, day_state)
-- cache (replaced by snapshot swap; every table has a version column)
sku, sku_uom_conversion, sku_barcode, sku_image/thumb_key, local_path NULL, tag, sku_tag, tax_rate, batch,
  lot, price_list_line, discount_rule,
control_price, mrp_policy, customer, operator(user_id, display_name, language, pin_hash, permissions_json),
  permission, count_task,
expected_drop, expected_transfer, inventory_policy, config, message_catalogue, assortment(sku_id,
  sell_through)
-- indexes: sku_barcode(barcode, symbology); batch(sku_id, batch_no); lot(location_id, batch_id);
  document(status, business_date); outbox(acked_at)
```

Aspect	Design
Encryption	SQLCipher with a key generated in the Android keystore at enrolment; the training namespace is a second database file with its own key.
Durability	PRAGMA journal_mode=WAL; PRAGMA synchronous=FULL set once; issuance uses a single Room transaction (Section 4); the app calls checkpoint at session close and before safe shutdown.
Migrations	Room migrations forward-only; run at start before the UI; a failed migration reports to the agent's self-test (doc 31 §5.4).

4. The issuance transaction

Restated from doc 18 §5.5 and doc 32 §3.1. The one function every document on the till goes through.

```
// data/IssuanceService.issue(draft: DocumentDraft, bundleBuilder: (Issued) -> Envelope, ctx:
  OperatorContext): Issued
db.withTransaction { // Room: BEGIN IMMEDIATE;
  synchronous=FULL
  val series = seriesFor(draft.docType, syncState) // till-position series, or location
    series if isPrimary else throw NotPrimaryTill
  val counter = counterDao.lock(series.id) // row read inside the transaction
  val number = counter.nextNumber; counterDao.update(series.id, number + 1)
```

```

val issued = draft.numbered(series.id, number, display(series,
    number)).withHash(Envelope.canonicalHash(draft))
    .withIssued(clock.now(), clock.local(), businessDate())
documentDao.insert(issued.header); lineDao.insertAll(issued.lines); tenderDao.insertAll(issued.tenders)
stateDao.insert(transition(issued, from = "DRAFT", to = issued.status, ctx))
issued.movements.forEach { m -> movementDao.insert(m.withSeq(sequence.next(issued.locationId))) }
auditDao.insert(audit(DOCUMENT_ISSUED, issued, ctx)); auditDao.insertAll(issued.extraAudit)
val seq = outboxDao.nextSeq() // dense device_seq
outboxDao.insert(OutboxRow(seq, envelope = bundleBuilder(issued).withDeviceSeq(seq))
issued.extraAudit.forEach { a -> outboxDao.insert(OutboxRow(outboxDao.nextSeq(), Envelope.audit(a))) }
cacheDao.applyLocalDeductions(issued.movements) // lot cache mirrors what was just
    sold (may go negative)
issued
} // COMMIT: all rows and the counter,
    or none
// Power loss anywhere inside leaves the database at the previous commit; the counter never advances without
    the document (spike 1 asserts)

```

5. Selling: lookup, engine, basket

```

// ui/sales/SalesViewModel (Compose state machine per doc 26 4.1)
onScan(raw): ScanResult = scanLayer.parse(raw) // Section 7
    lookup = catalogueLocal.lookup(result, locationId) // barcode -> (sku, uom, batch?) |
        gtin+lot -> batch | INTERNAL code -> sku(+weight|batch)
    ?: return showLookupFallback(result) // code entry / search by name in any
        script (trigram over the three names, ordered by shop language)
    addLine(LineInput(lookup.skuId, lookup.uom, qty = lookup.weightKg ?: 1, scannedBatchId = lookup.batchId,
        pickedBatchId = null))
addLine(i): r = PriceResolver.resolveLine(i, index, businessDate)
    when { r.needsPick -> showPicker(r.candidates) { chosen -> addLine(i.copy(pickedBatchId = chosen)) }
        !r.sellable -> toast(r.reason) // "price.no_list_line",
            "sku.inactive_no_sell_through"
        lotCache.wouldGoNegative(i) && !ctx.can("pos.negative_stock.acknowledge") -> supervisorPin {
            addLineAck(i) }
        lotCache.wouldGoNegative(i) -> inlinePrompt("stock.none_recorded") { addLineAck(i) }
        else -> basket.add(r) }
    basket = BasketResolver.resolve(basket.inputs, index, businessDate, tender = CASH_PREVIEW) // rules
        re-evaluated on every change
tender(kind, amount, ref): require(kind != CARD_REF || online, "tender.card_offline"); require(kind !=
    ACCOUNT || withinCap(amount), "tender.account_cap")
complete(): final = BasketResolver.resolve(..., tenderKind = dominantTender); receipt =
    IssuanceService.issue(receiptDraft(final, tenders), ::receiptBundle, ctx)
    printer.print(renderer.receipt(receipt)); drawer.kick(); customerDisplay.show(total, change); reset()
// index: PricingSnapshotIndex rebuilt on snapshot swap (Section 6); lookup tables indexed as in Section 3

```

6. Sync worker and snapshot application

Restated from doc 32 §3, 5, 6, 7 and 19A §8. A WorkManager periodic job (15 min) plus an immediate run after any issuance when online and on connectivity change; never blocks the UI.

```
// sync/UploadWorker
loop while outbox.hasUnacked():
    batch = outbox.nextBatch(maxEvents = 500, maxBytes = 2MB, fromSeq = syncState.lastAckedSeq + 1)    //
        contiguous, never splits a bundle
    resp = api.postBatch(deviceId, Batch(batchId = UUIDv7(), firstSeq, lastSeq, appVersion, snapshotVersion,
        clock, uptime, events, engineVersion))
    when (resp) {
        is Ack -> { outbox.markAcked(upTo = resp.lastAppliedSeq, at = now); syncState.lastAckedSeq =
            resp.lastAppliedSeq
                handleInstructions(resp.instructions); clockOffset = resp.clockOffsetMs;
                noteOutcomes(resp.outcomes) }    // QUARANTINED -> local anomaly audit
        is SeqMismatch(expected) -> { if (expected <= syncState.lastAckedSeq + 1) { resendFrom(expected) } else {
            localAnomaly("gap_ahead"); stop } }
        is Conflict409 -> backoff(); break    // another batch in flight (should
            not happen; single worker)
        is RateLimited(retryAfter) -> delay(retryAfter); continue
        is Auth401_403(revoke?) -> { if (revoke) executeRevoke(); stop }
        is Floor426 -> { syncState.floorState = REFUSED; stop }
        else -> backoffWithJitter(5s..10min); break
    }
outbox.purge(ackedBefore = now - 7.days)

// sync/SnapshotWorker (after upload, and on urgent hint in heartbeat)
head = api.snapshotHead(locationId); if (head.version <= syncState.snapshotVersion) return
delta = api.snapshot(locationId, since = syncState.snapshotVersion)    // Delta | FullPointer
staging.clear(); when (delta) { is Delta -> staging.applyRows(delta.rows); is FullPointer ->
    staging.loadFile(download(delta.url, resumable)) }
require(staging.verifyHashes(delta.manifest))
uiLock.betweenSales {    // waits for the current sale to
    complete or park
    db.withTransaction { cache.swapFrom(staging, holdPendingApplyFrom = businessDate); syncState.snapshotVersion
        = delta.version }
    index = PricingSnapshotIndex.build(cache); catalogueLocal.reindex()
}
thumbnails.fetchMissingLazily()
// day-open: when businessDate advances (all sessions closed or cut-off), cache.activatePending(applyFrom <=
    newBusinessDate); index rebuilt

// sync/HeartbeatWorker (5 min when online): post version, snapshotVersion, pendingCount, oldestPendingAge,
    battery, storage, peripherals, clock;
// apply instructions; if urgent flag -> SnapshotWorker.runNow()
// handleInstructions: REVOKE -> wipe caches, lock UI, keep outbox; FLOOR_NOTICE(grace) -> banner;
    REREQUEST_FROM(seq) -> outbox.unack(seq..);
// SERIES HOLDER CHANGE(series, mine) -> counter adopt/release (adopt: next_number from instruction;
    release: counter row removed)
```

7. Scan layer and peripheral layer

```
// peripherals/ScanLayer.parse(raw: RawScan): ScanResult
sym = raw.symbology ?: detect(raw.data)
when (sym) {
    EAN13, EAN8, UPCA -> ScanResult(sym, gtin = normalize14(raw.data))
    GS1_128, GS1_DATAMATRIX, GS1_QR -> { ais = Gs1.parse(raw.data /* element string or Digital Link URI */)
        ScanResult(sym, gtin = ais["01"], lot = ais["10"], expiry = ais["17"]?.toDate(), weightKg =
            ais["3103"]?.let { it / 1000.0 }) }
    INTERNAL_20_29 -> { code = raw.data.substring(2, 8); tail = raw.data.substring(8, 12)
        when (prefix) { in 20..24 -> ScanResult(sym, code = code, weightKg = tail / 1000.0); in 25..29 ->
            ScanResult(sym, code = code, batchRef = tail) } }
}
dedupe: a second read of the same GTIN within 700 ms from a different symbology (dual-marked pack) is dropped;
    one beep
// peripherals/Interfaces.kt (UnifiedPOS vocabulary)
interface Scanner { fun scans(): Flow<RawScan>; fun beep(); fun health(): Health }
interface PosPrinter { fun printRaster(bitmap: Bitmap): Result; fun cut(); fun health(): Health }
interface CashDrawer { fun kick(); fun isOpen(): Boolean }
interface Scale { fun readKg(): Result<BigDecimal>; fun health(): Health }
interface LineDisplay { fun show(top: String, bottom: String) }
interface LabelPrinter { fun printLabel(bitmap: Bitmap): Result }
// adapters: SunmiAdapter, PaxAdapter (placeholders until doc 14 DR-3), UsbHidScanner,
    SerialScale(protocolTable), Simulator (all interfaces; scripted events)
// selection by config at enrolment (terminal model -> adapter set); health checked at session open; failures
    reported in heartbeat
```

8. Receipt renderer, sessions, PIN, modes, training

```
// ui/print/ReceiptRenderer: Compose-free Canvas rendering to a Bitmap 576 px wide (80 mm at 203 dpi), Noto
// fonts, shop language with
// per-field English fallback (fallback lines get a small "EN" tag); sections per doc 26 3.4; then
// PosPrinter.printRaster (ESC/POS raster
// or vendor SDK); Z-report, day sales and session report use the same renderer with their own layouts.
// data/SessionService: open(float) -> one OPEN per position; drop(amount, reason) prints a slip;
// close(counted): expected = float + cashTenders
// - changeGiven - drops; variance = counted - expected; Z-report; session facts to outbox; abandons parked
// sales with REVIEW audit
// core/BusinessDataService: from sync_state.day_state; advances when all sessions closed or cut-off passes;
// triggers cache.activatePending
// security/PinAuth: verify(pin, operator.pin_hash) with Argon2id (m=64MB, t=3, p=2); 5 failures -> 15-min
// lockout (local + audit);
// permissions from operator.permissions_json (signed snapshot verified at swap); idle lock timers from
// config; operator switch audited
// ui/mode/ModeController: SALES <-> SHOP_MGMT; park-first; permission check; audit event; shop-mode screens
// hosted from feature modules
// m4grn, m5count, m5writeoff, m5repack, m5transfer (each renders its module's spec on the shared components
// and uses IssuanceService)
// primary-till rule: LOCATION-series issuance throws NotPrimaryTill -> the screen offers "save as draft"
// and shows the primary till's name
// training/TrainingController: opens the second database, prefix TRN-, watermark banner, simulator
// peripherals unless printer allowed; uploads
// go to the sandbox entity (device token carries training=true); exit restores the live database
// telemetry/Telemetry: crash reporter, heartbeat payload builder, local diagnostics screen (helpdesk
// boundaries: no connectivity tools)
```

9. Screens

Sales (zones per doc 30 4.3; 1280x800 reference)

```
+-----+
| Shop 07 . Till 1 . Nimal . si . [ONLINE 12:41] pending 0 | [scan/code/name _____] | |
| Basket | [thumb] Milk powder 400g 1 x 980.00 (MRP) 980.00 | [thumb] Milk powder 400g |
| [thumb] Red rice 1kg 2.350 kg x 220 517.00 (gazette) | EA . Rs 980.00 . MRP 980 (B2411A) |
| [thumb] Yoghurt 500ml 2 x 180 -18 (offer) 342.00 | stock 37 . exp 2027-03 |
| | [ - ] qty [ 1 ] [ + ] [Remove] |
| | +-----+
| | Subtotal 1,857.00 Offers -18.00 |
| | TOTAL Rs 1,839.00 |
| | [Cash] [Card ref] [Account] |
| | keypad 7 8 9 / 4 5 6 / 1 2 3 / 0 . |
| | +-----+
| [Park] [Resume 2] [Void] [Refund] [Reprint] [Supervisor] [Shop mode] |
+-----+
```

Picker (in the right zone): two ItemCards side by side with pack image, batch, MRP, expiry; tap selects; "Why am I seeing this?" -> one-line explanation
 Negative-stock prompt (inline, right zone): "Records show none in stock. Sell anyway?" [Sell] [Cancel]
 (supervisor PIN if not permitted)
 Blind close: "Count the cash in the drawer" [keypad] -> [Close] (expected figure never shown to the cashier);
 Z-report prints; supervisor view shows variance
 Shop-management home: tiles with icons and labels: Receive goods (2 expected) . Count (1 due) . Write-off .
 Repack . Transfer (1 in transit) . Expiry (4) . Reprint . Sync status

Screen	Source of spec	Notes
PIN / session / blind close	Doc 26 §3.5, §4.2	Float keypad; self-test results; Z-report
Sales, picker, negative-stock prompt, weigh	Doc 26 §3.2, §3.6; doc 30 §4.3	As sketched; ItemCard from M2 LookupResult; candidates from the engine
Park / resume, refund (same session), reprint	Doc 26 §3.3, §3.4, §4.1	Reason codes from the message catalogue
Shop-management home and hosted screens	Doc 24A §8 (GRN), doc 25A §8 (count, write-off, repack, transfer), doc 26 §3.7	Primary-till badge; draft hand-over
Sync and device status	Doc 26 §3.10, §9.7	Pending, last ack, snapshot age, floor, peripherals, battery, version
Training	Doc 26 §3.9	Loud badge, watermark
Web on the terminal	Doc 26 §3.7	Kiosk allow-listed URL; opens the back office for online-only functions

10. Central package m6pos

```
lk.coopfed.knoweb.m6pos/ @ApplicationModule(allowedDependencies =
  {"kernel", "m1party::query", "m2catalogue::query", "m3pricing::query", "m5inventory::query"})
ingest/ ReceiptBundleHook (called by the kernel bundle path): insert doc_receipt + tenders; re-validate
every line with the engine at the
reported snapshot and engine versions (kernel keeps snapshot versions for the change-log window);
flags -> line flag + ALERT/REVIEW audit;
publish receipt.issued/voided/refunded.v1 (M5 deducts, M7 posts account tenders)
SessionHook: till_session and cash_movement rows; variance beyond threshold -> REVIEW; publish
till_session.*.v1 (day-close trigger)
refund/ RefundCrossSessionHandler (@CommandHandler permission pos.refund.cross_session, online):
validates the original, issues a refund receipt
from the supervisor's till-position series (when at a till) or the location's refund series
(web), CREDITS link, SALE_REVERSAL via event
health/ DeviceHealthConsumer (till.heartbeat.v1 -> device_health upsert; device.last_seen);
SyncHealthQueries; StuckDevicesQuery (doc 31 fleet console)
query/ PosQueries: sessions, variances, receipts by number/search, sync health, device health
web/ PosController: refund, sessions, receipts, sync health
```

11. Tests

Layer	Cases	Target
Unit (JVM)	Scan layer (GS1 fixtures incl. Digital Link, 20–29 labels, dual marking); tender coverage and rounding; blind-close arithmetic; outbox batching rules (never split a bundle); snapshot staging and swap; apply_from activation; PIN lockout; mode guards; engine parity on the golden baskets (doc 23A)	All branches

Layer	Cases	Target
Power-loss harness (instrumentation, spike 1)	10,000 issuances with process kill and simulated power loss injected at each step of Section 4; assert: no document without its number, no number without its document, outbox dense, lot cache consistent; database opens clean every time	Zero loss or duplicate
Sync conformance	Doc 32 §11 thirteen cases against the central simulator; two-day backlog (about 800 receipts) drains; lost ack; central re-request; series holder change; revoke	All green
Peripherals (spike 3)	Each adapter against real devices on the candidate terminals; scale protocol table; simulator parity	Gating for the terminal choice
Rendering (spike 2)	Golden receipts in three languages and mixed fallback on each candidate printer; Z-report	Pixel-stable
Scenario	Flows 6.1–6.14 of doc 26; doc 13 scenarios 2 and 4; shop-mode bundles accepted by central (24A, 25A hooks)	All flows
Soak	48 hours disconnected on the reference terminal with power cuts every 4 hours; storage growth and battery measured	Under 200 MB; clean recovery
Usability (spike 7)	Two rounds with real cashiers in three languages: sale, refund, weigh, GRN; tap counts and error rates	Three taps to sell; targets set per round
Central	Re-validation flags not rejects; session variance REVIEW; cross-session refund; sync health queries; stuck devices	All cases

12. Tickets in build order (till track: T; central: C)

#	Ticket	Est. (days)	Done when
T-01	Room schema (till-owned and cache), encryption, migrations, DAOs; sequence and counter services	4	Schema tests; encryption verified
T-02	IssuanceService and the power-loss harness; bundle builders for RCT and shop-mode documents	4	Spike 1 green
T-03	Sync: UploadWorker, SnapshotWorker with staging swap, HeartbeatWorker, instructions; central simulator; conformance suite	7	Doc 32 §11 green
T-04	Engine adapter, PricingSnapshotIndex build on swap, catalogue local lookup and search (trigram over three names)	3	Parity and lookup tests
T-05	Scan layer and peripheral interfaces with Simulator; adapter scaffolds for two vendors	3	GS1 fixtures; simulator flows
T-06	Sales screen: basket, ItemCard, picker, negative-stock prompt, weigh, tenders, complete; park/resume; void; same-session refund; reprint	8	Three-tap sale on simulator; flows 6.2–6.7
T-07	Receipt renderer and printing; Z-report, day sales, session report	3	Spike 2 goldens
T-08	Sessions, cash drops, drawer, blind close, day-open activation	3	Flow 6.1, 6.9
T-09	PIN auth, idle locks, operator switch, permissions from the signed snapshot	2	Lockout and switch tests
T-10	Mode controller and shop-mode screens: GRN (24A), count, write-off draft, repack, transfer receipt (25A); primary-till rule; expiry list; sync status	8	Bundles accepted by central
T-11	Real peripherals on candidate terminals (spike 3); kiosk and device-owner cooperation with the agent (spike 6)	5	Gating tests pass on the chosen terminal
T-12	Training namespace; telemetry; diagnostics; safe shutdown	3	Soak test setup
T-13	Usability rounds 1 and 2 with real cashiers; fixes	4	Targets met
T-14	48-hour soak; release 1.0.0 signed with the dev key; APK to ring 0	3	Soak green
C-01	m6pos: receipt and session ingestion hooks with re-validation	3	Flags not rejects
C-02	Cross-session refund; sessions, receipts, variance queries	2	Flow 6.8
C-03	Device health consumer, sync health and stuck-device queries; contract freeze with M7, M8, doc 31	3	Fleet console data

Till total about 60 developer-days across three developers (T-03 and T-06 are the long poles; run them in parallel after T-02); central about 8. The terminal decision (doc 14 DR-3) must close before T-11; everything before it runs on the simulator and the generic emulator profile.

13. Open points: what to assume, whom to ask

Point	Assume	Decider	If different
Terminal model and vendor SDKs (doc 14 DR-3)	Generic 8-inch profile; two adapter scaffolds	Hardware procurement	Adapter implementation only
Cross-session refund series (doc 26 DR-1)	Supervisor's till-position series at a till; location refund series on the web	Federation accounts; kernel	Series definition in the kernel seed
Non-primary tills (DR-2)	Drafts only for LOCATION-series documents	Till track; Federation	Counter transfer semantics change (doc 32 §8)
Card reference mandatory, offline account cap, parked age, reprint limit, keyed weight (DR-3, 4, 5, 8)	Defaults of doc 26 §7	Federation	Config values
Receipt legal content (DR-6)	Doc 26 §3.4	Programme; tax advisor when available	Template fields
Customer display (DR-7)	Where a second screen exists	Federation	Adapter presence
Kiosk-exit procedure (DR-9)	Supervisor PIN plus time-boxed helpdesk code	DevOps, helpdesk lead	Doc 38
Thumbnail fetch policy	Lazy over Wi-Fi/Ethernet; on 4G only for the assortment's top 500 SKUs by sales	DevOps after spike 5	Config

Point	Assume	Decider	If different
Snapshot swap timing	Between sales; if a sale stays open over 10 minutes, the swap proceeds and the open sale re-resolves on next line change	Till track	—

End of document. 26A v0.1; the till repository's README, the power-loss harness and the conformance suite become the living version once merged.